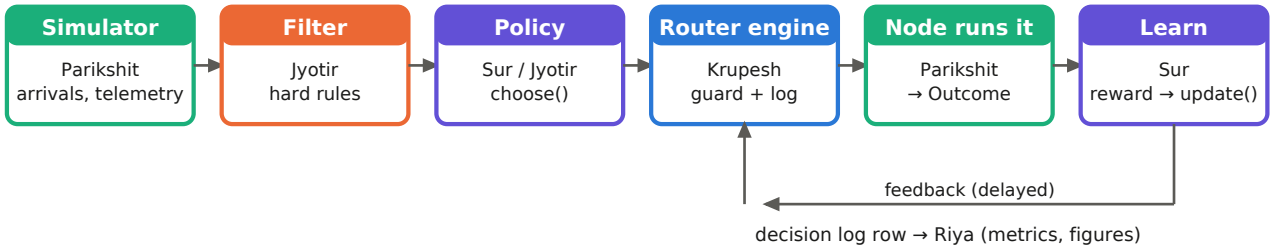
Interface & Dependency Handbook

AMS 560 · Group 20 · Change-Aware, Feasibility-First Scheduling for AI Inference · prepared by Krupesh (architecture & integration) · 2026-10-27

How to use this document. Every module talks to the router only through the contracts in Section 3. Each teammate has a chapter (Section 5) with what you receive from Krupesh, what Krupesh and others need from you, an acceptance checklist, and an **Agent brief** you can paste into your AI coding agent as its instructions. Values marked provisional need team approval (Section 7). Source of truth for code: `geo-inference-scheduler/Krupesh/` (not pushed to the shared repository yet).

1. The system in one loop

A real HTTP router (Krupesh) decides which simulated GPU site serves each LLM request. Hard safety rules (Jyotir) run first; a policy (Jyotir's baselines or Sur's LinUCB) chooses among the remaining sites; the simulated node (Parikshit) serves it and reports the outcome; the router computes the reward (Sur), updates the policy and writes one decision-log row that Riya turns into metrics and figures.

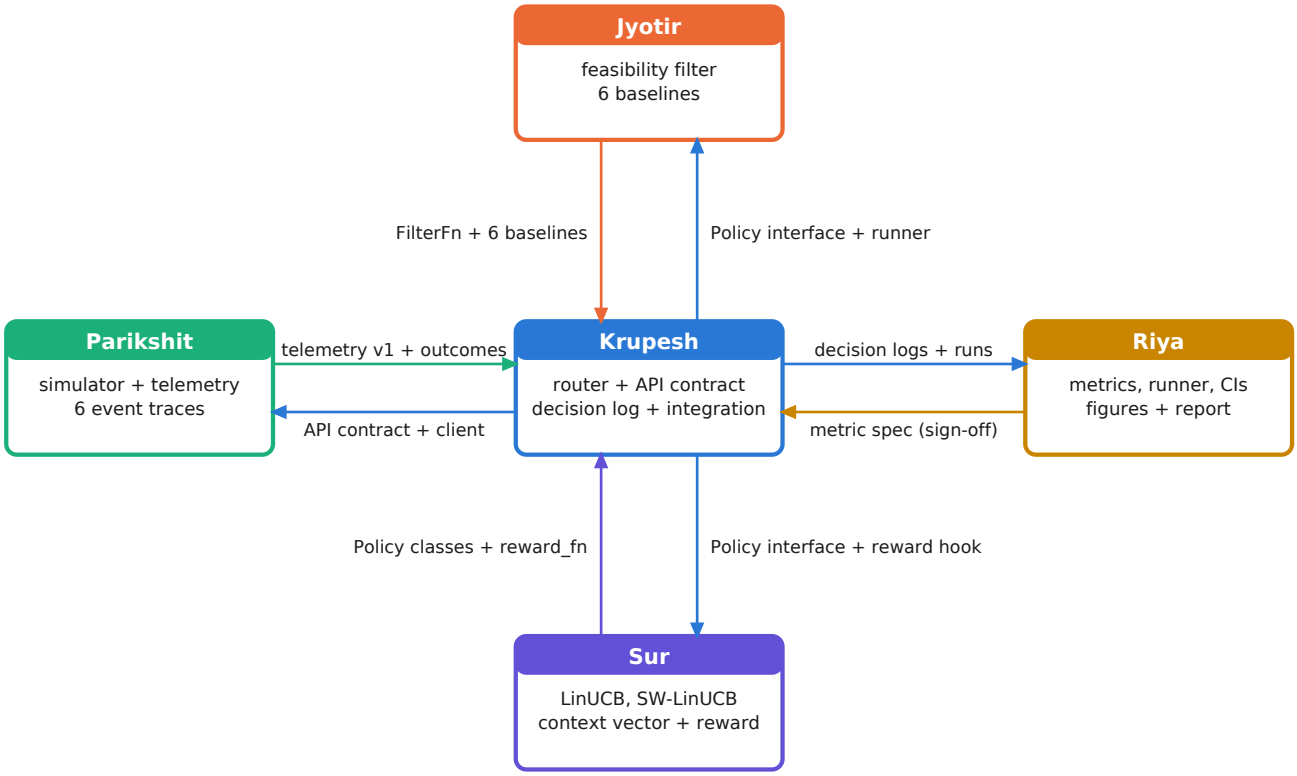


Module	Owner	Status today	Where it plugs into the router
Router service, API contract, decision log, integration	Krupesh	DONE (32 tests)	Krupesh/router_service, api_contract, decision_log, integration
Feasibility filter + 6 baselines	Jyotir	EXISTS, used unchanged	FilterFn + Policy via integration/jyotir_adapter.py
Simulator + telemetry + 6 traces	Parikshit	MISSING (mock used)	ingest_telemetry / route / feedback calls
LinUCB + SW-LinUCB + reward	Sur	MISSING (mocks used)	Policy subclass + reward_fn
Metrics, runner, CIs, figures	Riya	MISSING	reads decision logs only

2. Who depends on whom

Arrows point from the person who provides an artifact to the person who needs it. Colours follow the provider. Week numbers (W1–W4) follow the Work Distribution timeline.

2.1 Krupesh's router in the middle (arrow = provider → needer)



2.2 Dependencies between the other teammates

Provider	Needer	Artifact	Due
Parikshit	Jyotir, Sur	Telemetry schema v1 (field names + units): the filter rules and the context vector read it	end of W1
Parikshit	Riya	Six replayable event traces S1–S6 (+ real-data loaders)	S1/S2 W2, all W3
Jyotir	Sur	Filter callable: the bandit only ever scores filtered candidates	stub W1, full W2
Jyotir	Riya	Baseline decision logs on all six traces; rule/threshold table	W3
Sur	Riya	Window-sweep results ($W \in \{200, 500, 1000, 2000, 5000\}$); citations	W3
Riya	all	Adaptation-time definition (sign-off) and experiment-bundle format	spec W1, bundle W3

2.3 Full matrix — everyone

Person	Needs FROM (what, from whom, by when)	Provides TO (what, to whom, by when)
Krupesh	Telemetry schema v1 (Parikshit, W1) · feasibility filter interface (Jyotir, W1 stub / W2 full) · policy classes + reward (Sur, W2 plain / W3 windowed) · simulator that calls /telemetry and /feedback (Parikshit, W2)	API contract + Policy interface (everyone, W1) · decision-log format (Riya, W1) · running end-to-end system + logs (Riya, W2→) · weekly integration report (all)
Parikshit	API contract (Krupesh, W1)	Telemetry schema v1 (Krupesh, Jyotir, Sur, W1) · running simulator (W1 5 nodes) · 6 event traces (Riya, W3) · real-data loaders (Riya, W2)
Jyotir	Telemetry schema v1 (Parikshit, W1) · Policy interface (Krupesh, W1)	Filter module (Krupesh, Sur; W1 stub, W2 full) · 5–6 baselines (Krupesh, W2) · baseline logs on 6 traces (Riya, W3) · rule/threshold table (Riya, W3)
Sur	Telemetry schema v1 (Parikshit, W1) · filter (Jyotir, W1 stub) · Policy interface + Outcome type (Krupesh, W1)	LinUCB (Krupesh, W2) · SW-LinUCB (Krupesh, W3) · window sweep $W \in \{200,\ldots,5000\}$ (Riya, W3) · citations (Riya, W3)
Riya	Decision-log format (Krupesh, W1) · running system (Krupesh, W2) · 6 traces (Parikshit, W2–W3) · baseline logs (Jyotir, W3) · sweep results (Sur, W3)	Adaptation-time spec (all, W1) + code (W2) · experiment-bundle format (all, W3) · report + figures (all, W4)

3. Shared contracts (exact)

Owner: Krupesh for the API, Python interfaces and decision log; Parikshit for telemetry v1. Field names, types and units below are frozen v1 from Project Brief §10. Changing any of them needs a message to the whole group.

3.1 Telemetry record v1 (one per node per tick) — owner Parikshit

Field	Type	Unit / allowed values	Used by
schema_version	str	"1.0"	router (rejects other major versions)
t	float	simulation time, s (≥ 0)	router: telemetry age, local load accounting
node_id	str	unique per node	all
status	str	up | down | warming	filter (status rule), context (warming flag)
rtt_ms	dict[str, float]	ms per user region, ≥ 0	min-RTT, static-weighted, context
queue_len	int	≥ 0 waiting requests	least-load, power-of-two, context
running	int	≥ 0 in service	least-load, context
capacity	int	≥ 1 max concurrent	context (queue/capacity)
kv_used_frac	float	0..1 KV cache used	filter (memory), context
models_loaded	list[str]	e.g. ["llama-3-8b"]	filter (model)
power_headroom_kw	float	kW below the site cap	filter (power ≥ 2 kW)
temperature_c	float	°C	filter (< 85 °C), reward thermal term
energy_price_usd_kwh	float	USD/kWh ≥ 0	lowest-energy, static-weighted, reward
gpu_price_usd_hr	float	USD per GPU-hour ≥ 0	context, cost
kv_capacity_tokens	int | null	extension (optional): KV size in tokens	filter memory rule F2

3.2 Request (POST /route body)

Field	Type	Meaning
request_id	str	unique per run, e.g. r-000123
t_arrival	float	simulation time (s) the request arrives
user_region	str	key into rtt_ms, e.g. nyc
model	str	must be in models_loaded of the chosen node
input_tokens	int ≥ 1	prompt length
expected_output_tokens	int ≥ 1	router's estimate of output length
slo_class	str	interactive | batch
ttft_slo_ms	float > 0	TTFT promise (250 ms interactive). NOTE: not slo_ms

3.3 Decision (POST /route response)

```
{"request_id": "r-000123", "decision": "route" | "reject", "node_id": "us-east-1" | null,
 "policy": "jyotir:static_weighted", "feasible_nodes": ["us-east-1"],
 "filtered_out": {"us-west-2": "temperature_high"}, "reject_reason": null | "no_feasible_node"}
```

Filter reason strings (Jyotir's): node_down · node_warming · model_missing · memory_full · power_limit · temperature_high.

3.4 Outcome (POST /feedback body, sent when the request finishes)

Field	Type	Meaning
request_id	str	same id as /route
ttft_ms	float	network + queue + prefill
e2e_ms	float	ttft + decode (≥ ttft_ms)
slo_met	bool	ttft_ms ≤ ttft_slo_ms and not failed
energy_kwh	float ≥ 0	energy attributed to this request
cost_usd	float ≥ 0	energy cost + GPU rent share
failed	bool (ext.)	node went down while holding the request
t_complete	float (ext.)	simulation time the request finished
reward	float	filled by the ROUTER before policy.update(); senders leave it out

3.5 Python plug-in interfaces

Plug-in points

```
# Krupesh/router_service/policy_base.py          implemented by Jyotir (baselines) and Sur (bandits)
class Policy:
    def choose(self, request: Request, candidates: list[NodeState]) -> str: ...   # node_id
    def update(self, request: Request, node_id: str, outcome: Outcome) -> None: ...

# Krupesh/api_contract/types.py
FilterFn = Callable[[Request, list[NodeState]], FilterResult]   # Jyotir  -> allowed + {node: reason}
RewardFn = Callable[[Outcome, NodeState], float]                # Sur     -> reward for update()

# Krupesh/router_service/engine.py  (what the simulator calls, in simulated time)
router.ingest_telemetry(list[NodeState])   # every tick       (HTTP: POST /telemetry)
decision = router.route(Request)           # every arrival    (HTTP: POST /route)
reward = router.feedback(Outcome)          # on completion    (HTTP: POST /feedback)
```

Guarantees enforced by the router: (1) policies only ever see feasible nodes; (2) a choice outside them raises instead of routing; (3) empty feasible set → deliberate reject 'no_feasible_node'; (4) feedback is delayed and the reward is computed by the router before update(); (5) between telemetry ticks the router adds its own recent placements to queue_len/kv_used_frac.

3.6 Decision log v1 — 17 frozen columns, in order (owner Krupesh, consumer Riya)

#	Column	#	Column	#	Column
1	run_id	7	decision	13	e2e_ms
2	seed	8	node_id	14	slo_met
3	scenario	9	feasible_nodes (JSON)	15	energy_kwh
4	policy	10	filtered_out (JSON)	16	cost_usd
5	request_id	11	reject_reason	17	reward
6	t_arrival	12	ttft_ms		

Extensions after column 17: outcome_status (ok/failed/rejected/unfinished), t_complete, user_region, model, slo_class, ttft_slo_ms, input/expected tokens, chosen_node_status, telemetry_age_s, constraint_violation (must be null), true_state_violation, router_wall_ms (real time, never simulated).

4. What Krupesh's router needs from each teammate

These are the dependencies **on** Krupesh's work: until each item exists, the router runs with the labelled mock named in the last column.

From	Exactly what Krupesh's router needs	Where it plugs in	Status / due
Parikshit	Telemetry records with the 14 v1 fields (table §3.1), one per node per tick, sent via router.ingest_telemetry(list[NodeState]) or POST /telemetry	engine.ingest_telemetry() · POST /telemetry	MISSING → mock_simulator. Due W1–W2
Parikshit	One Outcome per routed request when it finishes or fails (failed=True), via router.feedback() or POST /feedback; simulated clock only	engine.feedback() · POST /feedback	MISSING → mock. Due W2
Parikshit	A driver loop that calls route() for every arrival in simulated time	reference: MockCluster.run() in mocks/mock_simulator.py	MISSING → mock. Due W2
Jyotir	Feasibility filter returning allowed + reasons	FilterFn via jyotir_adapter.py	DONE (original, unmodified). Fix F1/F2 requested
Jyotir	Baselines implementing Policy	registry.py: jyotir:	DONE (all 6 via adapter)
Sur	LinUCB and SW-LinUCB as Policy subclasses (choose/update, delayed feedback)	EXTRA_POLICIES in registry.py	MISSING → mock_policy. Due W2 / W3
Sur	reward_fn(outcome, node_at_dispatch) -> float (brief §6.3)	reward_fn of the engine (make_engine in app.py)	MISSING → mock_reward. Due W1–W2
Riya	Nothing for the router to run. Agreement on the decision-log format and adaptation-time spec	decision_log_format.md	format DONE; spec needs sign-off

5. One chapter per teammate

5.1 Jyotir — Feasibility filter + static baselines.

Status: EXISTS — connected to the router unchanged through Krupesh/integration/jyotir_adapter.py. His 14 tests pass; his folder is byte-identical to his last commit.

You receive from Krupesh

- **Policy interface** — Krupesh/router_service/policy_base.py (same signatures as his stub).
- **Typed contract** — Krupesh/api_contract/types.py (Request, NodeState, Outcome, FilterResult).
- **Runner for baseline logs** — Krupesh/integration/run_end_to_end.py; log format in decision_log_format.md.

Krupesh and the team need from you

- **F1 (safety):** warming nodes must still pass model/memory/power/temperature rules.
- **F2:** memory rule must include this request's tokens (needs kv_capacity_tokens).
- **Imports:** switch stubs.stub_policy → Krupesh's Policy/Outcome when the team merges.
- **Field name:** ttft_slo_ms (not slo_ms) — the adapter maps it until then.
- **To Riya:** baseline decision logs on all six scenarios; rule/threshold table.

Agent brief — paste this into your AI coding agent

AGENT BRIEF · Jyotir

```
ROLE: You are helping Jyotir (AMS 560 Group 20) finish the feasibility filter + static baselines.
REPO: geo-inference-scheduler/Jyotir/ (Jyotir's own code). Do not edit any other member's folder.
CONTRACT (owner Krupesh, file Krupesh/api_contract/types.py, Krupesh/router_service/policy_base.py):
  - Telemetry record fields (brief 10.1): schema_version, t, node_id, status(up|down|warming),
    rtt_ms{region:ms},
    queue_len, running, capacity, kv_used_frac(0..1), models_loaded[list], power_headroom_kw, temperature_c,
    energy_price_usd_kwh, gpu_price_usd_hr, optional kv_capacity_tokens.
  - Request fields: request_id, t_arrival, user_region, model, input_tokens, expected_output_tokens,
    slo_class, ttft_slo_ms   (use ttft_slo_ms, not slo_ms).
  - Policy: choose(request, candidates) -> node_id ; update(request, node_id, outcome) -> None.
TASKS:
  1. FIX F1 (safety): in feasibility_filter/filter.py::_check_node a "warming" node returns before the
     model/memory/power/temperature rules. Make "warming" only decide allow/deny (allow_warming_nodes);
     a warming node must still pass every other rule. Add a test: warming + 99 C -> rejected
     "temperature_high".
  2. F2: memory rule must include this request:
     kv_used_frac + (input_tokens + expected_output_tokens) / kv_capacity_tokens > kv_cache_max_frac
     -> "memory_full"; if kv_capacity_tokens is missing, keep the current kv_used_frac >= limit check.
  3. Keep reason strings exactly: node_down, node_warming, model_missing, memory_full, power_limit,
     temperature_high. Filter returns (allowed: list[dict], rejected: list[{node_id, reason}]).
  4. Replace "from stubs.stub_policy import Policy, Outcome" by Krupesh's Policy/Outcome when the team
     merges (Krupesh/router_service/policy_base.py, Krupesh/api_contract/types.py).
  5. Produce baseline decision logs on all 6 scenarios via Krupesh's runner (format: Krupesh/decision_log/
     decision_log_format.md) and give them to Riya. Write the rule/threshold table for the report.
ACCEPTANCE: pytest Jyotir/edge_case_tests passes; then in Krupesh/: pytest -o addopts= tests passes
  (test_known_issue_F1... must be updated to expect rejection after the fix);
  python -m integration.run_end_to_end prints constraint_violations: 0 and log_problems: [].
```

5.2 Parikshit — Node simulator, telemetry schema, six event traces, real-data loaders.

Status: MISSING — the router is tested against Krupesh/integration/mocks/mock_simulator.py (a labelled mock).

You receive from Krupesh

- **API contract** — Krupesh/api_contract/api_contract.md and schemas.py (HTTP), types.py (Python).
- **Reference call order** — mocks/mock_simulator.py::MockCluster.run.
- **HTTP client** — router_service/client.py::HttpRouter (same methods as the engine).

Krupesh and the team need from you

- **Telemetry v1** every tick: router.ingest_telemetry(list[NodeState]) or POST /telemetry.
- **route()** for every arrival, in simulated time.
- **Outcome** per routed request on completion/failure: router.feedback() or POST /feedback.
- **To Jyotir/Sur:** the frozen telemetry schema. **To Riya:** six replayable event traces.
- **Optional:** kv_capacity_tokens field (enables F2).

Agent brief — paste this into your AI coding agent

AGENT BRIEF · Parikshit

```
ROLE: You are helping Parikshit (AMS 560 Group 20) build the node simulator + telemetry + event traces.
REPO: put code in geo-inference-scheduler/Parikshit/ (or the agreed src layout). Do not edit others' folders.
CONTRACT (owner Krupesh): Krupesh/api_contract/types.py (NodeState, Request, Outcome),
  Krupesh/api_contract/api_contract.md (HTTP), Krupesh/integration/mocks/mock_simulator.py (reference loop).
BUILD:
  1. SimPy simulator of 5-10 heterogeneous nodes (brief section 5): state = rtt to each user region, queue,
     running, capacity, KV used, models loaded, power headroom, temperature, energy + GPU price,
     status up/down/warming.
  2. Latency: TTFT = T_network + T_queue + T_prefill; T_prefill = a_p + b_p*input_tokens;
     T_decode = out_tokens * t_tok * (1 + c*load); e2e = TTFT + T_decode. A node cannot start a request
     whose KV would overflow.
  3. Telemetry v1 exactly as brief 10.1 (field names + units above), one record per node per tick (1 s).
  4. Six replayable event traces, 600 s, change at t0 = 200 s: S1 stable, S2 burst x3 for 120 s,
     S3 price shift, S4 thermal throttle (node must really exceed the throttle point and can cross 85 C),
     S5 +80 ms RTT on a popular node, S6 node down at 200 s, back at 400 s warming (slower) then up.
  5. Fixed seed per run; generate the whole workload before the run so every policy sees the same requests.
INTEGRATION CALLS (simulated time only; never block on wall-clock):
  every tick:            router.ingest_telemetry(list[NodeState])    or POST /telemetry {"records":[...]}
  every arrival:         d = router.route(Request)                   or POST /route
  if d.decision=="route": submit to node d.node_id
  when a request ends:   router.feedback(Outcome(...))               or POST /feedback
                         failed=True if the node went down while holding it; slo_met = ttft_ms <= ttft_slo_ms
ACCEPTANCE: replace MockCluster in Krupesh/integration/run_end_to_end.py with your simulator; run it
  in-process
  and over HTTP; decision log must validate (log_problems: []); unit tests for latency + telemetry validity.
```

5.3 Sur — LinUCB, sliding-window LinUCB, context vector, reward, window sweep, citations.

Status: MISSING — the router is tested with mocks/mock_policy.py and mocks/mock_reward.py (labelled mocks).

You receive from Krupesh

- **Policy interface** — router_service/policy_base.py; registry hook EXTRA_POLICIES.
- **Reward hook** — RewardFn = (Outcome, NodeState) -> float, called by the engine before update().
- **Delayed-feedback pattern** — mocks/mock_policy.py (cache by request_id).

Krupesh and the team need from you

- **LinUCB** (W2) and **SW-LinUCB** (W3) as Policy subclasses registered as sur:linucb / sur:sw_linucb.
- **reward_fn** with weights in a config file.
- **To Riya:** window-sweep results (W ∈ {200, 500, 1000, 2000, 5000}) and citations.

Agent brief — paste this into your AI coding agent

AGENT BRIEF · Sur

```
ROLE: You are helping Sur (AMS 560 Group 20) implement LinUCB, sliding-window LinUCB, context vector, reward.
REPO: geo-inference-scheduler/Sur/ (or agreed layout). Do not edit others' folders.
CONTRACT (owner Krupesh): implement Krupesh/router_service/policy_base.py::Policy
  class MyPolicy(Policy):
      def choose(self, request: Request, candidates: list[NodeState]) -> str   # node_id from candidates
      def update(self, request: Request, node_id: str, outcome: Outcome) -> None
  reward function signature: reward_fn(outcome: Outcome, node_at_dispatch: NodeState) -> float
RULES THE ROUTER ENFORCES:
  - candidates are ALREADY filtered (feasible only); returning anything else raises RuntimeError.
  - feedback is DELAYED (seconds of sim time) and may arrive out of order: in choose(), store the context
    vector x of the chosen node keyed by request.request_id; in update(), pop it and use outcome.reward
    (the router computes reward with your reward_fn BEFORE calling update).
  - failed requests arrive with outcome.failed=True and slo_met=False.
ALGORITHMS (brief 6.4-6.5): per node a: A_a = lambda*I, b_a = 0; theta = A^-1 b;
  score = theta.x + alpha*sqrt(x^T A^-1 x); choose max; update A += x x^T, b += r x.
  Sliding window: keep last W observations; on eviction A -= x x^T, b -= r x. Sweep W in
  {200, 500, 1000, 2000, 5000}. Window global vs per-node is a TEAM decision.
CONTEXT (brief 6.2): input length, expected output, SLO class, rtt to user region, queue, running,
  KV used, energy + GPU price, temperature, warming flag; FIXED scaling constants from config (no data
  leakage). REWARD (6.3): r = w1*[SLO met] - w2*latency - w3*energy_cost - w4*thermal, terms in 0..1.
REGISTER: Krupesh/router_service/registry.py: EXTRA_POLICIES["sur:linucb"] = lambda seed, **p:
  LinUCB(seed=seed, **p)
  and pass your reward_fn where mock_reward is used (app.py make_engine, integration/run_end_to_end.py).
ACCEPTANCE: in Krupesh/: python -m integration.run_end_to_end --policy sur:sw_linucb -> log_problems: [],
  constraint_violations: 0; unit tests: closed-form theta check, window add/remove restores A and b.
```

5.4 Riya — Metrics incl. adaptation time, experiment runner, bootstrap CIs, ablations, figures, report.

Status: MISSING — not needed for the router to run; she consumes decision logs.

You receive from Krupesh

- **Decision-log format v1** — Krupesh/decision_log/decision_log_format.md; reader/validator in writer.py.
- **Running system** — integration/run_end_to_end.py (in-process or HTTP) producing one log per run.

Krupesh and the team need from you

- **Adaptation-time spec** for team sign-off (and its noise floor measured on S1).
- **Experiment-bundle format** (config, seed, git commit, log, summary) — W3.
- **Report + figures** regenerated only from bundles — W4.

Agent brief — paste this into your AI coding agent

AGENT BRIEF · Riya

```
ROLE: You are helping Riya (AMS 560 Group 20) build metrics, adaptation time, experiment runner, CIs, figures.
INPUT CONTRACT (owner Krupesh): one decision log per run, format Krupesh/decision_log/decision_log_format.md:
  17 frozen columns in order: run_id, seed, scenario, policy, request_id, t_arrival, decision, node_id,
  feasible_nodes(JSON), filtered_out(JSON), reject_reason, ttft_ms, e2e_ms, slo_met, energy_kwh, cost_usd,
  reward; plus outcome_status (ok|failed|rejected|unfinished), t_complete, router_wall_ms, ...
  Read with Krupesh/decision_log/writer.py::read_log; validate with validate_log.
COMPUTE ONLY FROM LOGS (never from simulator memory):
  goodput = SLO-met requests / duration; p95/p99 TTFT and e2e over completed rows; rejection rate;
  energy and cost per SLO-met request.
  Adaptation time (brief 8, team sign-off needed): G(t) goodput in 1 s bins, 5 s moving average;
  G* = mean G over [t0+60, t0+120); adaptation = first t >= t0 with G >= 0.9 G* for 10 s in a row, minus t0;
  never -> "did not recover"; always report G*, deepest dip, lost goodput vs oracle.
  Report the metric's NOISE FLOOR: compute it on S1 (no change) too.
EXPERIMENTS: every policy x 6 scenarios x >= 10 seeds, identical workloads/traces per seed; 95% bootstrap
  CIs; paired differences per seed; tune hyper-parameters on separate seeds (not the reported ones);
  bundle per run = config + seed + git commit + decision log + summary table.
ACCEPTANCE: metrics reproduce on a hand-built log with known answers; figures regenerate from bundles only.
```

6. How to verify an integration (any teammate)

Commands

```
cd geo-inference-scheduler/Krupesh
pip install -r requirements.txt
pytest -o addopts= tests                          # must stay green (32 tests today)
python -m integration.run_end_to_end              # closed loop with Jyotir's original code
python -m integration.run_end_to_end --policy mock:learning --outage
uvicorn router_service.app:app --port 8000 &      # real HTTP service, docs at /docs
python -m integration.run_end_to_end --mode http --url http://127.0.0.1:8000

Expected in every summary:  "log_problems": []   and   "constraint_violations": 0
```

- Implement only against `api_contract/types.py` and `router_service/policy_base.py`.
- Register your policy in `router_service/registry.py` (EXTRA_POLICIES) or pass your function as filter_fn / reward_fn.
- Delete the matching file in `integration/mocks/` once your module is connected.
- Never edit another member's folder; propose changes in a pull request reviewed by one teammate.

7. Open decisions that need the team

#	Decision	Provisional default	Owner to decide
1	Adaptation-time settings (90 %, 10 s, G* window 60–120 s)	brief values; 5 s trailing average	all (Riya leads)
2	5 vs 10 nodes in main experiments	5 nodes	all
3	Window counted globally or per node	global (Cheung et al.)	Sur
4	Reward weights	w = 1.0 / 0.3 / 0.2 / 0.1	Sur
5	Exact filter limits; fix F1/F2; extra queue/capacity rule?	85 °C, 2 kW, KV 0.90 (Jyotir's file)	Jyotir
6	Reason strings	Jyotir's (temperature_high, ...)	Jyotir + Krupesh
7	Optional telemetry field kv_capacity_tokens	optional extension	Parikshit
8	Report format; calendar dates	Markdown draft; not set	all

8. Timeline (Work Distribution §7) — ✔ = done in Krupesh's build

Week	Krupesh	Parikshit	Jyotir	Sur	Riya
1	API contract, log format, router skeleton ✔	telemetry schema; 5 nodes; tests	rules + config ✔	context vector + reward draft	metrics spec; adaptation-time code
2	router + filter + baselines end to end ✔	latency model; real traces	5 baselines through filter ✔	plain LinUCB	runner + first baseline numbers
3	LinUCB plugged in; integration fixes	6 event traces	baselines on 6 traces; edge tests	SW-LinUCB; W sweep	full runs, CIs, ablations
4	final review; design section	simulator section	re-runs; baselines section	method section	figures, report, slides

9. Glossary

Term	Meaning
TTFT	time to first token = network + queue + prefill
SLO / goodput	the promise (TTFT ≤ 250 ms) / requests per second that kept it
Telemetry	numbers each node reports every tick (Section 3.1)
Feasible node	passes every hard rule; the only nodes a policy may choose

Term	Meaning
Delayed feedback	the outcome arrives when the request finishes, seconds after the decision
Mock / adapter	temporary stand-in for a missing module / wrapper that lets original code fit the interface
Simulated vs wall time	experiments use only simulated time; router_wall_ms measures real HTTP time

Prepared with AI assistance (Claude Code) under Krupesh's direction; contracts verified by the test suite in Krupesh/tests. Team review required before submission.